CALMS-SAVANT: análise técnica e implementação verificável

Cleilson Elias de Sousa - arquitetura de referência

Análise e versão de software: 11 de novembro de 2026. Documento de origem: SAVANT: Sparse Adaptive Vector Alignment for Continual Learning and Persistent Agent Safety, PDF de 19 páginas, datado no texto de 18 de maio de 2026.

1. Parecer

A contribuição científica investigável é tratar a consolidação de experiências em parâmetros como uma operação sujeita a verificação e possível rejeição. O problema é relevante mesmo sem exigir originalidade absoluta: aprendizagem persistente precisa equilibrar aquisição útil, retenção e resistência à integração de informação adversarial. Uma implementação aberta torna essa proposta mensurável e permite identificar quais mecanismos realmente ajudam.

O PDF contém material suficiente para implementar os componentes principais, mas não fornece, isoladamente, evidência verificável para reproduzir os números relatados ou as garantias mais fortes. O software entregue é uma implementação de referência de novembro de 2026, com correções e extensões explícitas. Não é uma reconstrução certificada do código que teria produzido os resultados do artigo.

O parecer é favorável à publicação como software de pesquisa, com alegações limitadas ao que foi efetivamente implementado e executado. A formulação matemática precisa ser revisada antes de servir como garantia de segurança.

2. O que foi examinado e o que falta

O texto integral local foi extraído e lido, incluindo os apêndices A-E. A página 3 também foi renderizada para conferir o pipeline e a interpretação da Eq. (2). A documentação de referência de transporte ótimo, LoRA e integração foi consultada em fontes primárias. Não foi realizada uma revisão sistemática de anterioridade nesta tarefa.

Material	Situação
PDF fornecido	Acesso integral local; 19 páginas
Resultados PSR 4,8% e FRS 92,3%	Afirmações presentes no PDF; não reproduzidas
Código original e ambiente dos experimentos do artigo	Não fornecidos no material examinado
PPP-Bench original, prompts e divisões	Não fornecidos para execução independente
Logs das cinco sementes e saídas dos juízes	Não fornecidos
Pesos e checkpoints do backbone 8B	Não fornecidos nem executados
Código novo e testes desta entrega	Implementados e executados conforme validation/

Ausência desses materiais impede verificar as alegações; não estabelece, por si só, que sejam falsas. O hash do PDF identifica os bytes analisados e não comprova uma data de divulgação pública ou anterioridade intelectual.

3. Deriva geométrica: três objetos diferentes

A Eq. (2) compara a distribuição de cada âncora antes e depois de uma proposta de atualização. O apêndice E.2 descreve, em vez disso, o transporte entre âncoras históricas e os embeddings do lote corrente. A segunda comparação mistura novidade do conteúdo com mudança do modelo e não estima necessariamente a deformação causada pelo passo de treinamento.

O código mantém IDs de âncoras e compara cada prompt consigo mesmo, executado pelo modelo completo nos dois estados. Não permite que uma permutação de comportamentos entre prompts seja escondida pela igualdade da nuvem agregada. Quando a extração depende somente de uma base congelada e ignora os adaptadores atualizados, a deriva medida é trivialmente zero e não pode governar esses pesos.

O custo 1-cos do apêndice está no intervalo [0,2], não [0,1]. Além disso, distância cosseno não satisfaz em geral a desigualdade triangular. O código usa como padrão a distância euclidiana entre vetores unitários dividida por dois, e disponibiliza o custo cosseno normalizado como alternativa identificada.

Transporte entrópico, custo do plano entrópico e W1 exato são objetos distintos. Um plano regularizado pode ter autocusto positivo. O padrão implementado utiliza a divergência Sinkhorn com autocustos removidos, mantendo também o modo de custo de transporte. Esses nomes e valores aparecem separadamente na API.

O solver opera em log e verifica as marginais. Cinquenta iterações, como sugeridas no PDF, não garantem convergência em toda distribuição ou epsilon. O limite padrão foi ampliado e uma falha de convergência impede a aprovação numérica do passo.

4. Penalidade imune e supressão

A Eq. (3) penaliza oposição de uma trajetória aos nós protegidos. A Eq. (E.5) seleciona o elemento de maior similaridade de um lote. Essa seleção pode ocultar um elemento contraditório quando o lote também contém um alinhado. A implementação calcula a penalidade por amostra e só aplica nós de domínio e escopo explícitos.

Para um grafo finito com direções normalizadas, cada hinge está entre zero e um. Consequentemente, a penalidade é limitada pela soma dos pesos dos nós aplicáveis. O limite em que essa penalidade tende ao infinito, usado nos apêndices D/E, não ocorre nesse domínio com número e pesos fixos de nós. A exponencial é positiva para risco finito; underflow de ponto flutuante não é uma garantia matemática.

Mesmo um deslocamento pequeno e constante pode acumular deriva ao longo do tempo. Clipping dá um limite ao gradiente dos fatores por passo, mas não prova segurança do comportamento. O código combina atenuação com rejeição por limiares finitos, mede o passo realmente aplicado e compara periodicamente com a época inicial.

A Eq. (6) também precisa de convenção explícita para o sinal do gradiente. O código executa descida, com sinal negativo. Na integração opcional é usado SGD direto; simplesmente multiplicar o gradiente antes do Adam pode ter seu efeito cancelado ou alterado pela normalização adaptativa dos momentos.

5. Ranks e separação de domínios

A regra rank = rank-base / lambda pode resultar em rank fracionário, maior que a dimensão da matriz ou incompatível com o orçamento total. É necessário arredondamento, limites e política de alocação explícitos. Nesta versão cada banco recebe um rank inteiro, respeitando dimensões e orçamento.

Posto um não significa rigidez absoluta: uma matriz de rank um ainda pode produzir uma mudança funcional grande. Também não há garantia de que um banco rotulado como privacidade só afete privacidade. O domínio é fornecido pelo chamador, e os bancos podem interferir em comportamentos compartilhados.

O núcleo NumPy treina fatores reais A e B com gradientes analíticos conferidos por diferenças finitas. A base permanece congelada e adaptadores não são mesclados automaticamente. Na integração PEFT, a seleção de um adaptador por domínio é explícita e as âncoras são medidas em todas as rotas configuradas.

6. Consenso de sementes: contraexemplo ao argumento apresentado

O Teorema 1 não é estabelecido pelas hipóteses fornecidas. A condição de maioria benigna não fornece automaticamente uma margem de separação em relação à média dos scores contaminados. Média e desvio padrão são influenciados pelos próprios outliers que deveriam identificar. A distância a uma referência limpa também não determina, sozinha, a distância agregada entre candidatos.

Considere três sementes com um único nó: v1 = v2 = (1,0) e v3 = (-1,0). Com custo euclidiano normalizado, as distâncias entre as duas primeiras são zero e as distâncias à terceira são um. As medianas excluindo a diagonal são 0,5; 0,5; 1. A média é 2/3 e o desvio padrão populacional é aproximadamente 0,2357. Para alpha = 2, o limiar média + alpha*desvio é aproximadamente 1,1381. Nenhuma semente é excluída, embora a contaminada seja minoria. Escolhendo delta-detect = 0,9, a separação unitária é estritamente maior que esse valor e 2 < 0,9/0,2357.

Esse exemplo satisfaz a condição de sensibilidade expressa no texto, mas a regra de poda não elimina a semente divergente. Reescalar uniformemente os custos para o intervalo [0,2] não muda essa conclusão. Isso invalida a inferência de poda necessária sob essas condições; não significa que toda forma de consenso seja inútil.

A hipótese sobre pares benignos não controla diretamente todos os scores adversariais, a média contaminada ou as dependências entre distâncias. Contaminar poucos nós também dilui a distância média entre grafos. A união de eventos não requer independência, embora a prova a mencione; o problema principal está no limite individual não derivado.

O código preserva a regra original como **paper_mean_std**, e oferece medoid/MAD como heurística alternativa. A alternativa rejeita esse exemplo particular, mas não recebe uma garantia bizantina geral. IDs semânticos precisam ser curados: máxima similaridade vetorial não comprova equivalência de significado, e uma correspondência gulosa pode nem ser bijetiva.

7. Arquivo e replay

Arquivar padrões rejeitados é útil para investigar recorrência e construir exemplos defensivos. Um vetor normalizado, porém, não preserva automaticamente texto, proveniência detalhada ou um alvo comportamental correto. O nome TCS não substitui uma medida demonstrada de compressão ou preservação topológica.

Nesta versão o arquivo contém assinaturas vetoriais e metadados limitados. Deduplicação combina proximidade e domínio; o excesso é descartado com critério temporal e severidade explícitos. Busca exata é o padrão verificável; HNSW é opcional, aproximado e não foi executado nesta entrega.

O benchmark usa uma penalidade de recorrência com peso 0,5 como extensão operacional. O padrão do núcleo mantém esse peso em zero, permitindo estudar a proposta sem essa extensão. Uma assinatura adversarial nunca se transforma automaticamente em um alvo de treino aprovado.

O exemplo de replay defensivo implementa loss contrastiva com um alvo seguro explicitamente fornecido e um alvo negativo. Isso demonstra o mecanismo numérico; o pipeline de obter e verificar alvos seguros para linguagem natural continua sendo trabalho experimental necessário.

8. Auditoria histórica e limites de cobertura

Proximidade à inicialização preserva consistência com uma raiz, mas não detecta que a própria raiz já era inadequada. Portanto, o monitor histórico não resolve a contaminação de maioria na inicialização nem o comprometimento das âncoras.

O código preserva a época 0 separadamente, reavalia candidatos de recuperação, restaura um dentro de metade do limiar e mantém o treinamento congelado até revisão registrada. Checkpoints frequentes, por si sós, não garantem a existência de uma versão segura; a raiz só é elegível se continuar reprodutível e confiável.

Um conjunto finito de âncoras também não cobre necessariamente todas as direções comportamentais. O contraexemplo executável da entrega modifica uma entrada não observada e inverte sua saída, mantendo deriva zero em todas as âncoras. Esse é um limite observacional real; não basta acrescentar um nome a um segundo filtro para demonstrar que todas as vulnerabilidades restantes foram resolvidas.

A normalização ainda remove informação de amplitude. Modelos podem alterar logits, confiança, escala ou leitura da representação sem uma rotação detectável pela medida escolhida. Avaliações de saída e probes comportamentais independentes devem acompanhar o monitor geométrico.

9. Resultados executados nesta entrega

O benchmark sintético utiliza vetores de dimensão oito, LoRA realmente treinável, cinco sementes, cem épocas e cinco registros por época, incluindo exatamente um ataque em cada grupo. Sete variantes compartilham os mesmos dados por semente. Os rótulos de ataque são usados apenas pelo avaliador.

Variante	Falha nos quatro probes protegidos	Cosseno factual médio	Deriva histórica média
Replay sem governança	0,25	0,99932	0,27598
Atenuação isolada	0,25	0,99924	0,27602
Completo	0,00	0,99919	0,06139
Sem identidade	0,00	0,99923	0,05923
Sem TCS	0,00	0,99919	0,06321
Sem auditoria	0,00	0,99919	0,06139
Ranks uniformes	0,00	0,99879	0,05239

Os números são médias das cinco sementes. Desvios amostrais, SEM, curvas por época, configurações, aceitação de dados benignos/adversariais e contadores de decisões estão no JSON bruto. Falha de 0,25 significa um de quatro probes sintéticos, não 25% de sucesso em prompts adversariais de linguagem natural. O cosseno factual é outra medida sintética, não o FRS do artigo.

Nenhuma execução deste benchmark ficou congelada. Nesse cenário, a remoção da identidade ou da auditoria não piorou a métrica de falha protegida. Portanto, esses dados não sustentam necessidade causal desses componentes. O TCS teve diferença pequena na deriva, sem diferença nessa métrica de falha. Não se transferem para este experimento as ablações fortes relatadas pelo PDF.

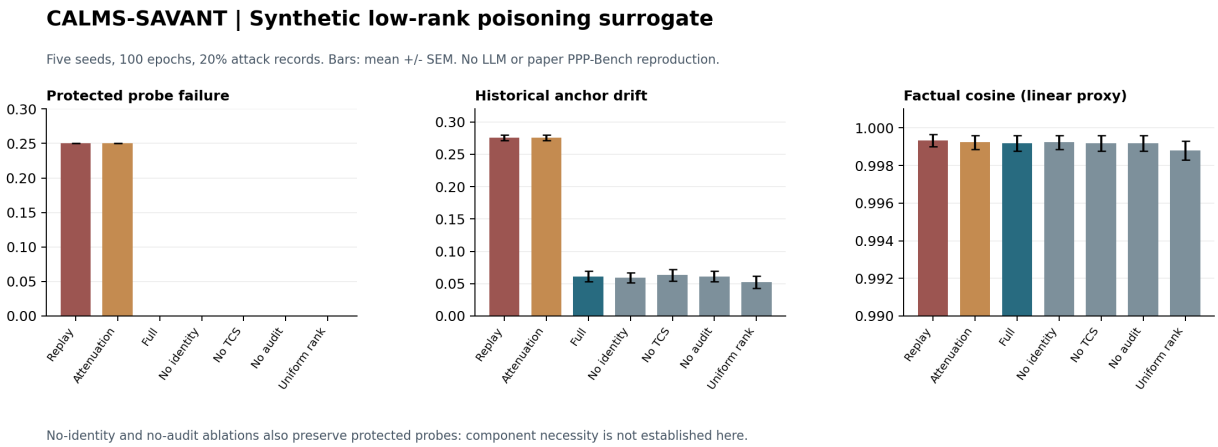


Figura 1. Surrogate linear; barras indicam médias e SEM entre cinco sementes. O eixo do cosseno factual é ampliado. Não constitui reprodução do PPP-Bench.

As verificações compreenderam 42 testes, com 40 executados com sucesso e dois omitidos por dependências opcionais ausentes: HNSW e PyTorch. A solução Sinkhorn foi comparada com um programa linear independente SciPy em um problema pequeno. Gradientes de MSE e loss contrastiva foram conferidos por diferenças finitas. Também foram verificados rejeição sem mudança de pesos, checkpoints, continuação após reabertura, checksum, capacidade do arquivo e contraexemplos. O registro final de validação acompanha o pacote.

10. Custo e alegações de latência

Colocar a verificação fora do caminho de atendimento reduz o custo síncrono, mas não demonstra latência literalmente zero. Adaptadores não mesclados, contenda por GPU, snapshots, publicação de estado e I/O têm custos concretos. Esta versão não implementa um servidor concorrente de inferência.

Com nuvens de n amostras por âncora, o custo inclui construir matrizes de custo, iterar o transporte e executar o modelo nas âncoras. A divergência pode exigir três problemas de OT por comparação. O tamanho do arquivo é O(M*d) em bytes vetoriais, além de metadados, grafo e checkpoints; O(M) só descreve o número de itens para dimensão fixa. HNSW não fornece busca logarítmica garantida em qualquer distribuição. Essas distinções devem substituir garantias de custo irrestritas.

11. Próximos experimentos úteis

Primeiro, recuperar ou reconstruir publicamente o protocolo PPP-Bench: prompts, geração, sementes, divisões, referência limpa, juízes e registros de desacordo. Sem isso, denominar um novo teste como reprodução seria impreciso.

Segundo, executar um transformer local pequeno com o treinador opcional, fixando revisões das dependências e do modelo. Medir retenção, adaptação benigna, transferência de ataques, precisão e revocação do filtro, deriva e custo por passo. Separar calibração dos limiares e avaliação final.

Terceiro, criar ataques nos quais as ablações sejam discriminantes: contradição relevante ao prior mas não às âncoras, recorrência que o arquivo reconheça, acúmulo abaixo do limiar local e mudanças na leitura sem alteração das âncoras. Comparar também rejeição por perda, limiar de norma de atualização e replay validado, além de EWC implementado de fato.

Quarto, estudar sensibilidade a bootstrap, escala, normalização, custo de OT, epsilon, cobertura e contaminação de âncoras. Separar garantia numérica, consistência com um prior e segurança comportamental. São propriedades diferentes.

O objetivo prático da publicação é permitir esse trabalho incremental e reprodutível. A arquitetura permanece uma hipótese de governança de plasticidade; o código permite testá-la e localizar suas falhas.

12. Referências e nível de acesso

Documento principal: Cleilson Elias de Sousa, SAVANT: Sparse Adaptive Vector Alignment for Continual Learning and Persistent Agent Safety, 18/05/2026. Acesso integral local ao PDF fornecido. Identificação dos bytes em validation/source_document.json; o PDF não é redistribuído neste pacote.

Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport. NeurIPS. Consulta ao registro e resumo oficiais; não é uma revisão exaustiva. https://proceedings.neurips.cc/paper_files/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html

Feydy et al. (2019). Interpolating between Optimal Transport and MMD using Sinkhorn Divergences. PMLR 89:2681-2690. Consulta ao registro e resumo oficiais. https://proceedings.mlr.press/v89/feydy19a.html

Hu et al. (2021/2022). LoRA: Low-Rank Adaptation of Large Language Models. Consulta ao registro e resumo oficiais arXiv; não houve reprodução de seus experimentos de transformer nesta tarefa. https://arxiv.org/abs/2106.09685

PyTorch. Documentação oficial de SGD. Acesso à documentação da API e equações. https://docs.pytorch.org/docs/main/generated/torch.optim.sgd.SGD_class.html

Hugging Face PEFT. Documentação oficial de modelos e controle de ranks. Consulta às assinaturas públicas e documentação de adaptadores. https://huggingface.co/docs/peft/main/package_reference/peft_model https://github.com/huggingface/peft/blob/main/docs/source/package_reference/lora.md

As referências de software foram consultadas em 11/11/2026. Dependências opcionais são explicitamente separadas das dependências e verificações do núcleo.